

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LFCX: py2fr = Kleiner Python Refresher

ÜBUNG LFCX:py2fr:01

- ☐ Bringen Sie ein Python-Hello-World auf Ihrem Rechner zum Laufen. [→ ZP:Sheet:03]

(→ [ex-py2fr-a-hello-world.py](#))

ÜBUNG LFCX:py2fr:02

- ☐ Bringen Sie das Programm von [→ ZP:Sheet:02] auf Ihrem Rechner zum Laufen.

(→ [ex-py2fr-b-nw-validation.py](#))

ÜBUNG LFCX:py2fr:03

- ☐ Schreiben Sie mit den Mitteln aus obigem Programm [→ ZP:Sheet:02] eine Funktion / Programm,
 - ☐ die eine IPv4-Adresse samt einer zu ihr passenden Netzmaske und eine beliebige andere IPv4-Adresse nimmt
 - ☐ diese von Kommandozeilenparametern überschreiben lassen kann
 - ☐ und die berechnet, ob beide Adressen in demselben Netz sind

Nutzen Sie bei der Berechnung **Bitweise Operatoren**, nicht fertige Abfragefunktionen aus Bibliotheken wie z.B. [ipaddress](#). Sie dürfen aber gern deren andere Funktionen nutzen, um IPv4-Strings in Zahlen (und v.v.) umzuwandeln.

ÜBUNG LFCX:py2fr:04

- ☐ Fügen Sie eine logische Bedingung ein, sodass der Test nur gestartet wird, wenn die beiden IPv4-Adressen nicht gleich sind.
-

ÜBUNG LFCX:py2fr:05

- ☐ Überführen Sie die bisherige funktionale Lösung in eine objektorientierte Lösung (*Schreiben Sie eine Klasse mit Python-Konstruktor und getter- und setter-Methoden*)
 - ☐ Fügen Sie eine Methode `isMember` hinzu, die eine fremde IPv4-Adresse nimmt und `true` oder `false` zurückgibt, je nachdem, ob die übergebene IPv4-Adresse zu dem Netz gehört oder nicht
 - ☐ Erlauben Sie im Main-Teil Ihres Programms, dass die zu testende IPv4-Adresse an der Kommandozeile an Ihr Programm übergeben wird
-

ÜBUNG LFCX:py2fr:06

- ☐ Python kennt keine typisierten Variablen. (Die Typisierung ist dynamisch). Warum ist das ungünstig bei Funktionen/Methoden, deren Parameter in der Definition per Default einen Wert zugewiesen bekommen?
-